In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay,
precision_score, recall_score, f1_score, accuracy_score
```

In [2]:
```
df = pd.read_csv('nba_naive.csv')
```

In [3]:
```
display(df.head())
```

	fg	3p	ft	reb	ast	stl	blk	tov	target_5yrs	total
0	34.7	25.0	69.9	4.1	1.9	0.4	0.4	1.3	0	266.4
1	29.6	23.5	76.5	2.4	3.7	1.1	0.5	1.6	0	252.0
2	42.2	24.4	67.0	2.2	1.0	0.5	0.3	1.0	0	384.8
3	42.6	22.6	68.9	1.9	0.8	0.6	0.1	1.0	1	330.6
4	52.4	0.0	67.4	2.5	0.3	0.3	0.4	0.8	1	216.0

In [4]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1340 entries, 0 to 1339
Data columns (total 11 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   fg            1340 non-null   float64
 1   3p            1340 non-null   float64
 2   ft            1340 non-null   float64
 3   reb           1340 non-null   float64
 4   ast           1340 non-null   float64
 5   stl           1340 non-null   float64
 6   blk           1340 non-null   float64
 7   tov           1340 non-null   float64
 8   target_5yrs   1340 non-null   int64
 9   total_points  1340 non-null   float64
 10  efficiency    1340 non-null   float64
dtypes: float64(10), int64(1)
memory usage: 115.3 KB
```

In [5]:
```
df.shape
```

Out[5]:
```
(1340, 11)
```

In [6]:
```
df.describe()
```

Out[6]:

	fg	3p	ft	reb	
count	1340.000000	1340.000000	1340.000000	1340.000000	1340.0(
mean	44.169403	19.149627	70.300299	3.034478	1.5505
std	6.137679	16.051861	10.578479	2.057774	1.4711(
min	23.800000	0.000000	0.000000	0.300000	0.0000(
25%	40.200000	0.000000	64.700000	1.500000	0.6000(
50%	44.100000	22.200000	71.250000	2.500000	1.1000(
75%	47.900000	32.500000	77.600000	4.000000	2.0000(
max	73.700000	100.000000	100.000000	13.900000	10.600(

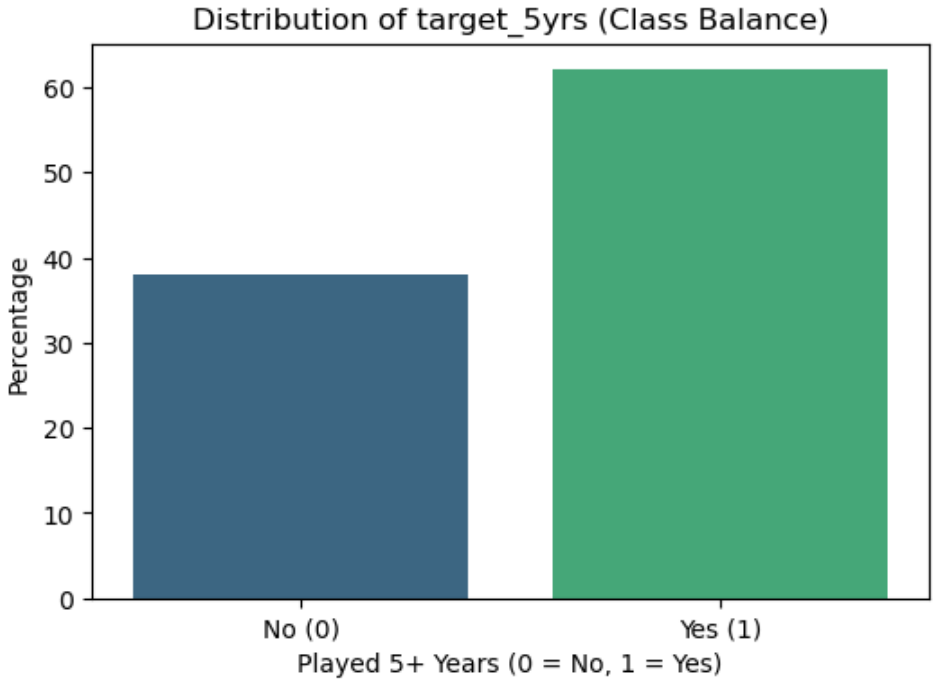
In [7]:
```
X = df.drop('target_5yrs', axis=1)
y = df['target_5yrs']
```

In [8]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42, stratify=y)

print(f'X_train shape: {X_train.shape}')
print(f'X_test shape: {X_test.shape}')
print(f'y_train shape: {y_train.shape}')
print(f'y_test shape: {y_test.shape}')
```

```
X_train shape: (1072, 10)
X_test shape: (268, 10)
y_train shape: (1072,)
y_test shape: (268,)
```

In [9]:
```
class_balance = y.value_counts(normalize=True) * 100
print("Class Balance for 'target_5yrs':")
display(class_balance)

plt.figure(figsize=(6, 4))
sns.barplot(x=class_balance.index, y=class_balance.values,
palette='viridis')
plt.title('Distribution of target_5yrs (Class Balance)')
plt.xlabel('Played 5+ Years (0 = No, 1 = Yes)')
plt.ylabel('Percentage')
plt.xticks(ticks=[0, 1], labels=['No (0)', 'Yes (1)'])
plt.show()
```

```
Class Balance for 'target_5yrs':


target_5yrs
1    62.014925
0    37.985075
Name: proportion, dtype: float64


  C:\Users\LENOVO
  THINKPAD\AppData\Local\Temp\ipykernel_19560\1060767397.py:6:
  FutureWarning:

  Passing `palette` without assigning `hue` is deprecated and
  will be removed in v0.14.0. Assign the `x` variable to `hue`
  and set `legend=False` for the same effect.

    sns.barplot(x=class_balance.index, y=class_balance.values,
  palette='viridis')
```


Distribution of target_5yrs (Class Balance)
Percentage
0
10
20
30
40
50
60
No (0)
Yes (1)
Played 5+ Years (0 = No, 1 = Yes)

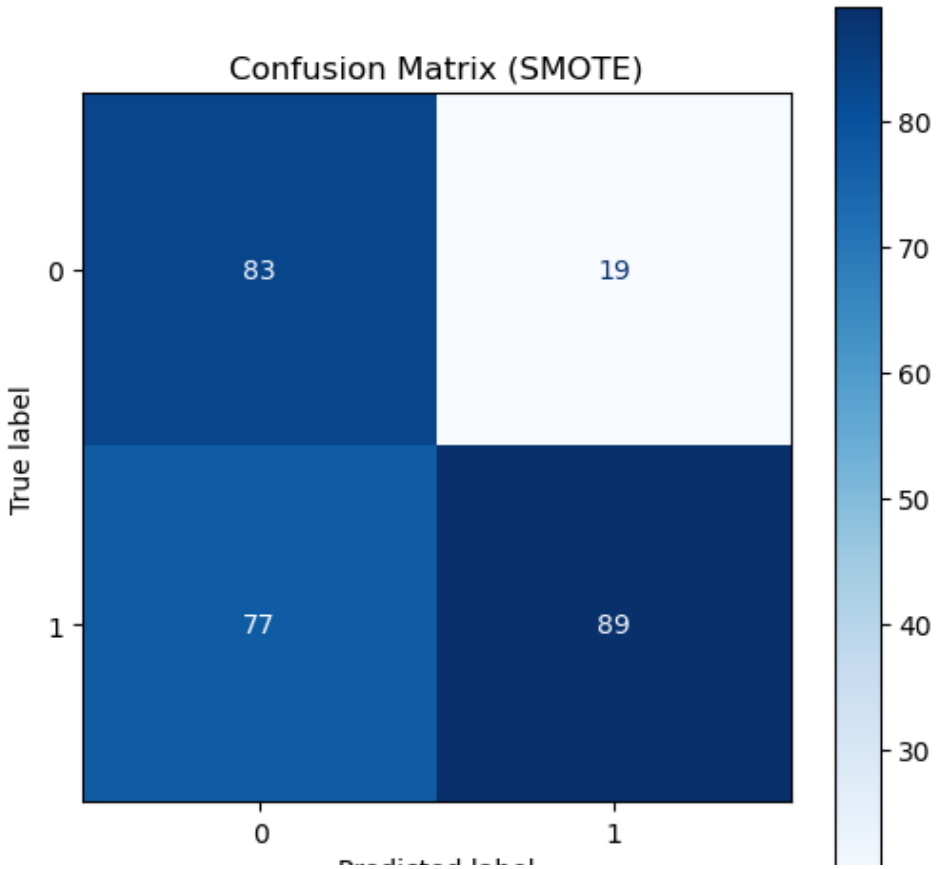
In [10]:
```
from imblearn.over_sampling import SMOTE

smote = SMOTE(random_state=42)
X_train_resampled, y_train_resampled = smote.fit_resample(X_train,
y_train)

print("Shape of X_train before SMOTE:", X_train.shape)
print("Shape of y_train before SMOTE:", y_train.shape)
print("Shape of X_train after SMOTE:", X_train_resampled.shape)
print("Shape of y_train after SMOTE:", y_train_resampled.shape)

print("\nClass balance in y_train before SMOTE:")
display(y_train.value_counts(normalize=True) * 100)

print("\nClass balance in y_train after SMOTE:")
display(y_train_resampled.value_counts(normalize=True) * 100)
```

```
Shape of X_train before SMOTE: (1072, 10)
Shape of y_train before SMOTE: (1072,)
Shape of X_train after SMOTE: (1330, 10)
Shape of y_train after SMOTE: (1330,)

Class balance in y_train before SMOTE:


target_5yrs
1    62.033582
0    37.966418
Name: proportion, dtype: float64


Class balance in y_train after SMOTE:


target_5yrs
1    50.0
0    50.0
Name: proportion, dtype: float64
```

In [11]:

```
gnb_smote = GaussianNB()
gnb_smote.fit(X_train_resampled, y_train_resampled)
y_pred_smote = gnb_smote.predict(X_test)

accuracy_smote = accuracy_score(y_test, y_pred_smote)
precision_smote = precision_score(y_test, y_pred_smote)
recall_smote = recall_score(y_test, y_pred_smote)
f1_smote = f1_score(y_test, y_pred_smote)

print(f"Accuracy after SMOTE: {accuracy_smote:.4f}")
print(f"Precision after SMOTE: {precision_smote:.4f}")
print(f"Recall after SMOTE: {recall_smote:.4f}")
print(f"F1-Score after SMOTE: {f1_smote:.4f}")

cm_smote = confusion_matrix(y_test, y_pred_smote)
display_smote = ConfusionMatrixDisplay(confusion_matrix=cm_smote,
display_labels=gnb_smote.classes_)

fig, ax = plt.subplots(figsize=(6, 6))
display_smote.plot(ax=ax, cmap='Blues')
plt.title('Confusion Matrix (SMOTE)')
plt.show()
```

```
Accuracy after SMOTE: 0.6418
Precision after SMOTE: 0.8241
Recall after SMOTE: 0.5361
F1-Score after SMOTE: 0.6496
```




Predicted label
20

Explanation of the Independence Assumption:

This assumption states that all features (e.g., `fg`, `3p`, `ft`, `reb`, `ast`, etc.) are independent of each other, given the class label (`target_5yrs`). In simpler terms, it assumes that the presence or absence of one feature does not affect the presence or absence of any other feature for a given outcome (whether a player plays 5+ years or not).

Realism for Basketball Stats (e.g., correlation between points and minutes played):

In the context of basketball statistics, the independence assumption is almost certainly **unrealistic**.

Consider the following:

1. **Correlation between points and minutes played:** Players who play more minutes (which would likely be a feature in a more comprehensive dataset) naturally tend to score more points. If 'minutes played' and 'total_points' were both features, they would be highly correlated, violating the independence assumption.
2. **Correlation between assists and turnovers:** Players who handle the ball more to create assists are also more likely to commit turnovers. These features are inherently linked.
3. **Correlation between field goal percentage and total points:** A higher field goal percentage (`fg`) often contributes directly to `total_points`. These are not independent.
4. **Rebounds and blocks:** Certain positions or player types excel in both rebounding and blocking shots, indicating a correlation between these defensive stats.

Because many basketball statistics are interconnected and influence each other (e.g., offensive stats often correlate with each other, and defensive stats with each other, and even offensive with defensive for all-around players), the assumption of independence is largely violated. For example, `total_points` is calculated from other features like `fg`, `3p`, `ft`, meaning it's directly dependent on them.

Impact of the violation:

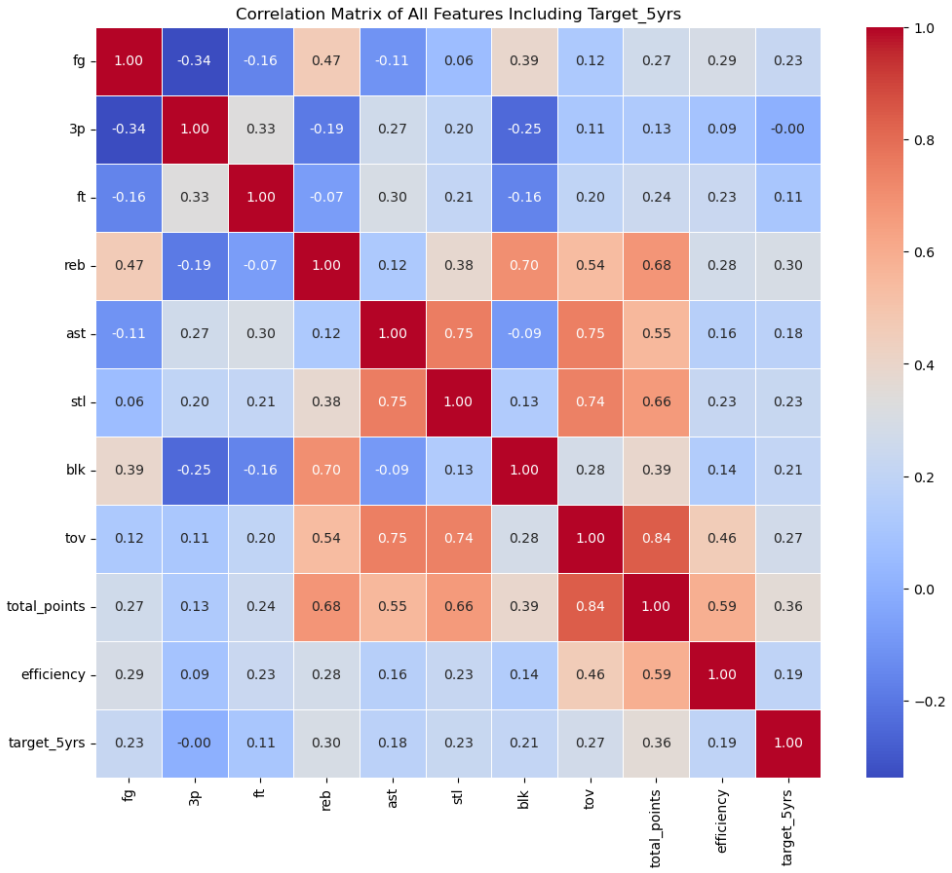
Despite this theoretical violation, Naive Bayes models can still perform well in practice. This is often because what matters for classification is not necessarily the accurate estimation of probabilities themselves, but rather the ranking of probabilities for different classes. If the independence assumption leads to a consistent (though inaccurate) ranking, the classifier can still make correct predictions.

However, understanding this limitation is crucial. When features are highly correlated, the model might overweight the contribution of these features, as it treats them as separate pieces of evidence. For instance, if 'total_points' and 'fg' are both strong

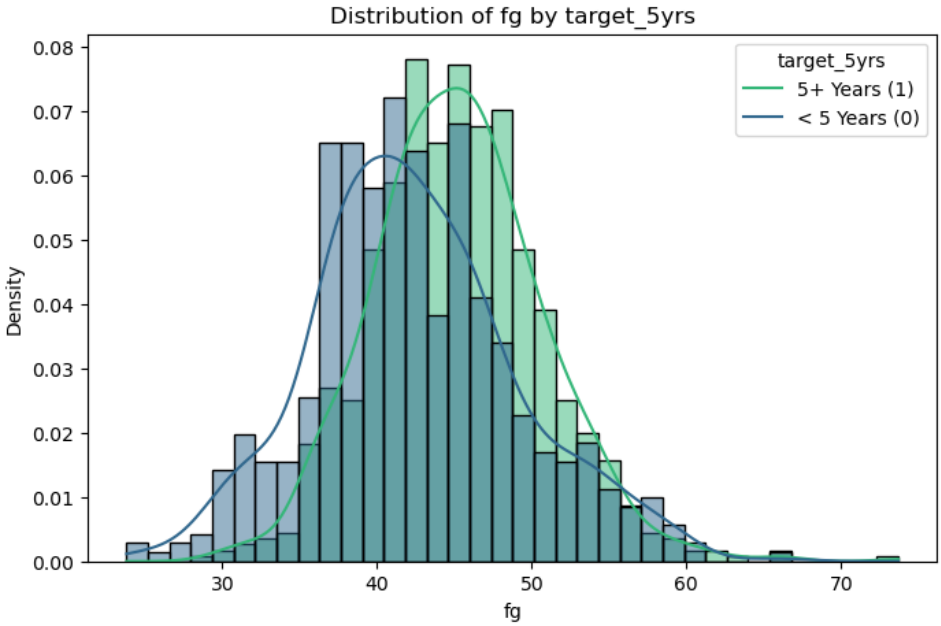
predictors and highly correlated, the model might count their predictive power twice, which isn't ideal but can still yield good classification performance if the relative weights are maintained.

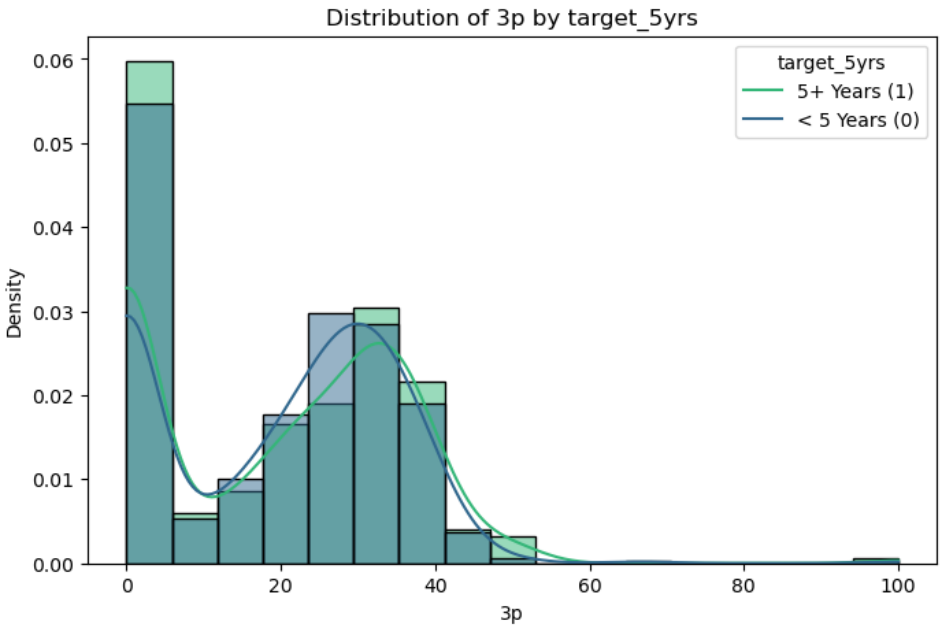
In summary, while the independence assumption is a pragmatic simplification that makes Naive Bayes computationally efficient, it is not truly reflective of the complex, interdependent nature of basketball statistics.

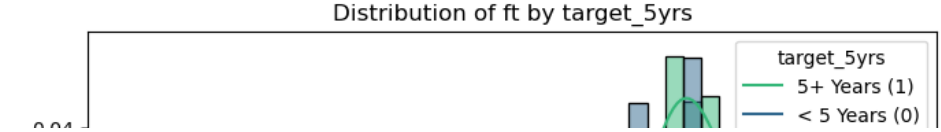
In [12]:
```
df_full_corr = pd.concat([X, y], axis=1)

plt.figure(figsize=(12, 10))
sns.heatmap(df_full_corr.corr(), annot=True, cmap='coolwarm', fmt='.2f',
linewidths=.5)
plt.title('Correlation Matrix of All Features Including Target_5yrs')
plt.show()
```

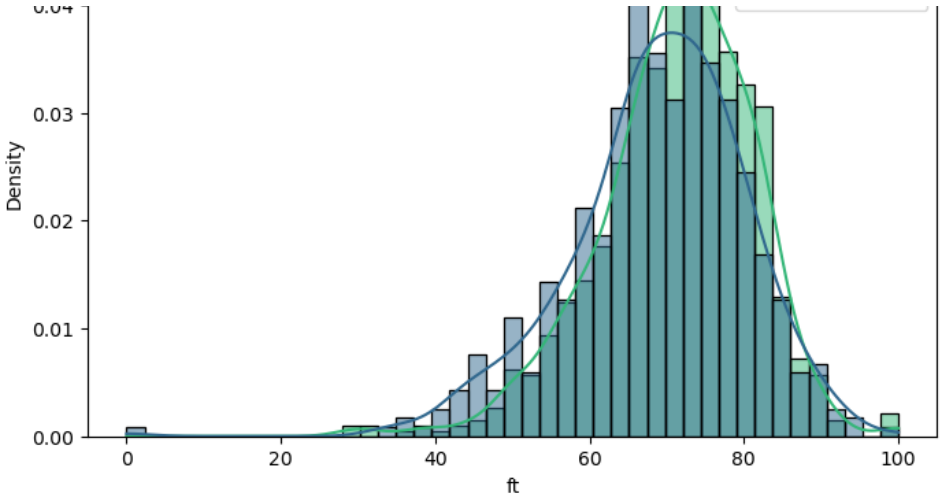


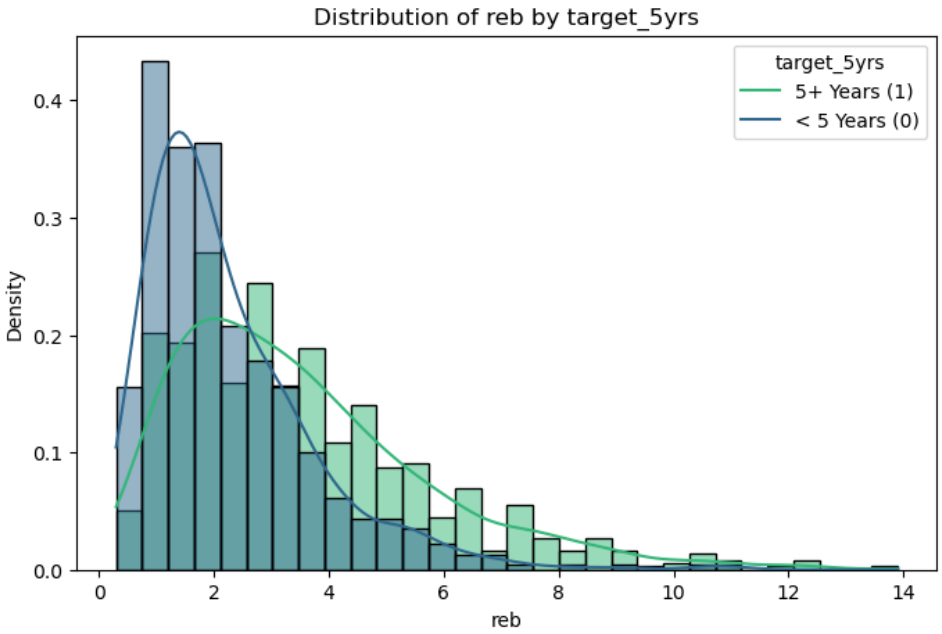
In [14]:
```
for column in X.columns:
    plt.figure(figsize=(8, 5))
    sns.histplot(data=df, x=column, hue='target_5yrs', kde=True,
palette='viridis', stat='density', common_norm=False)
    plt.title(f'Distribution of {column} by target_5yrs')
    plt.xlabel(column)
    plt.ylabel('Density')
    plt.legend(title='target_5yrs', labels=['5+ Years (1)', '< 5 Years
(0)'])
    plt.show()
```

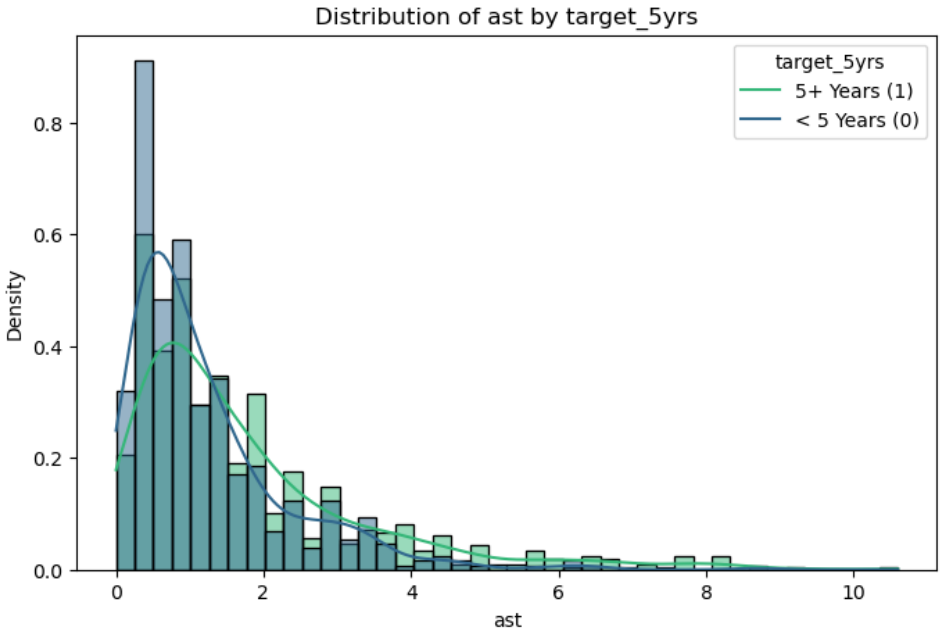


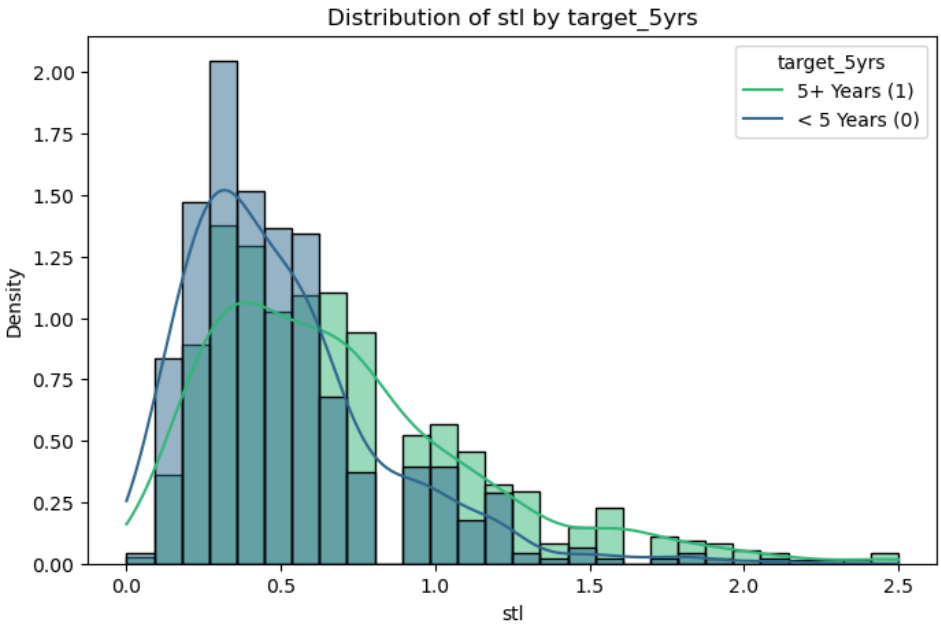


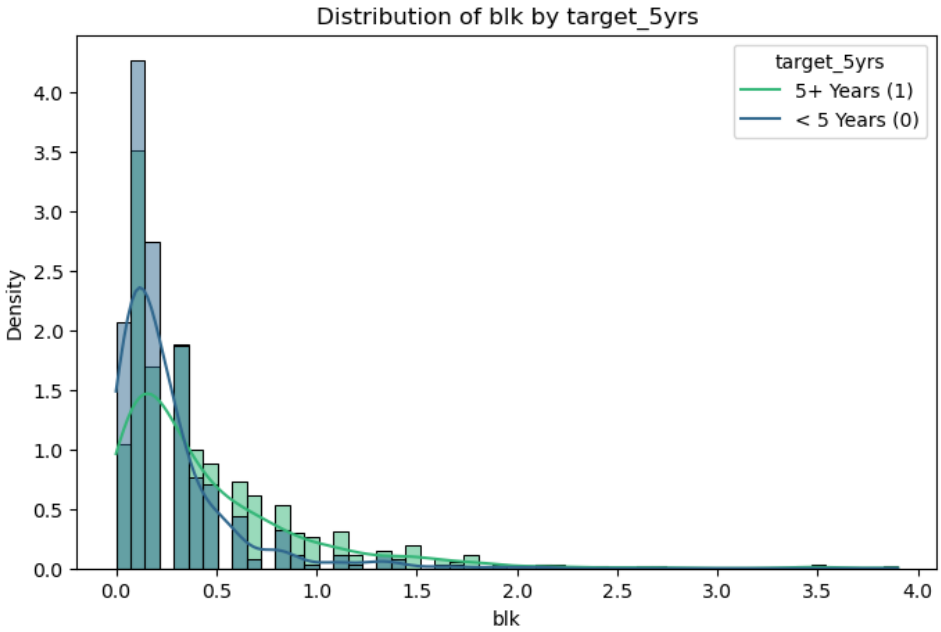


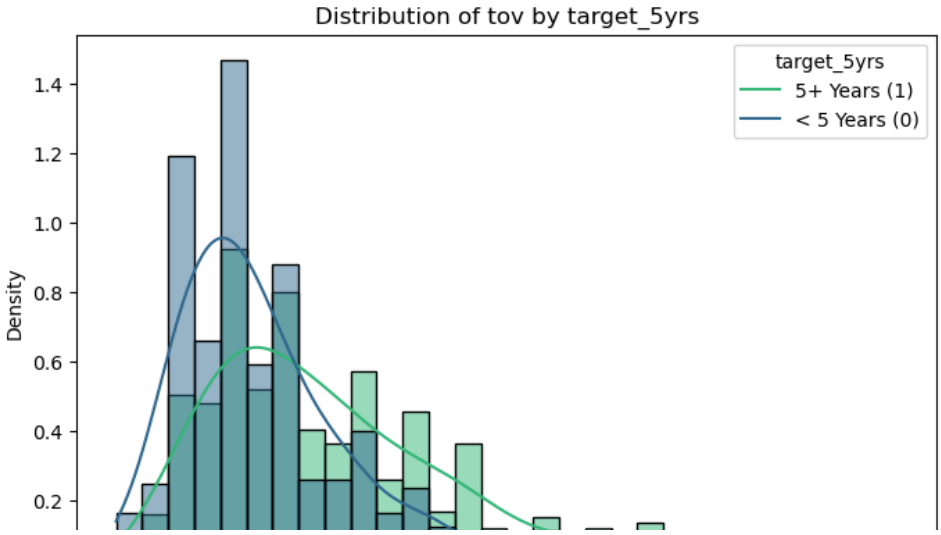

Density
0.00
0.01
0.02
0.03
0.04
0
20
40
60
80
100
ft

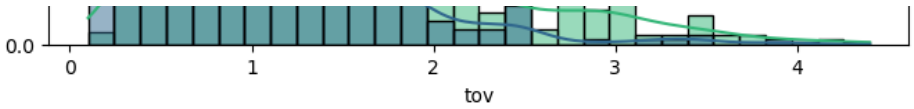

Distribution of reb by target_5yrs
target_5yrs
5+ Years (1)
< 5 Years (0)
Density
0.0
0.1
0.2
0.3
0.4
0
2
4
6
8
10
12
14
reb

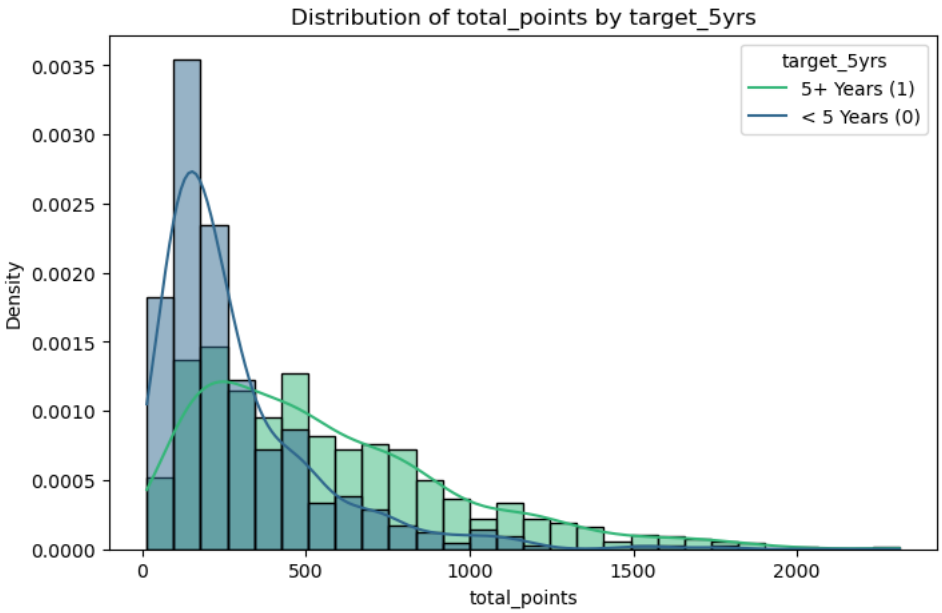

Distribution of ast by target_5yrs
target_5yrs
5+ Years (1)
< 5 Years (0)
Density
0.0
0.2
0.4
0.6
0.8
0
2
4
6
8
10
ast

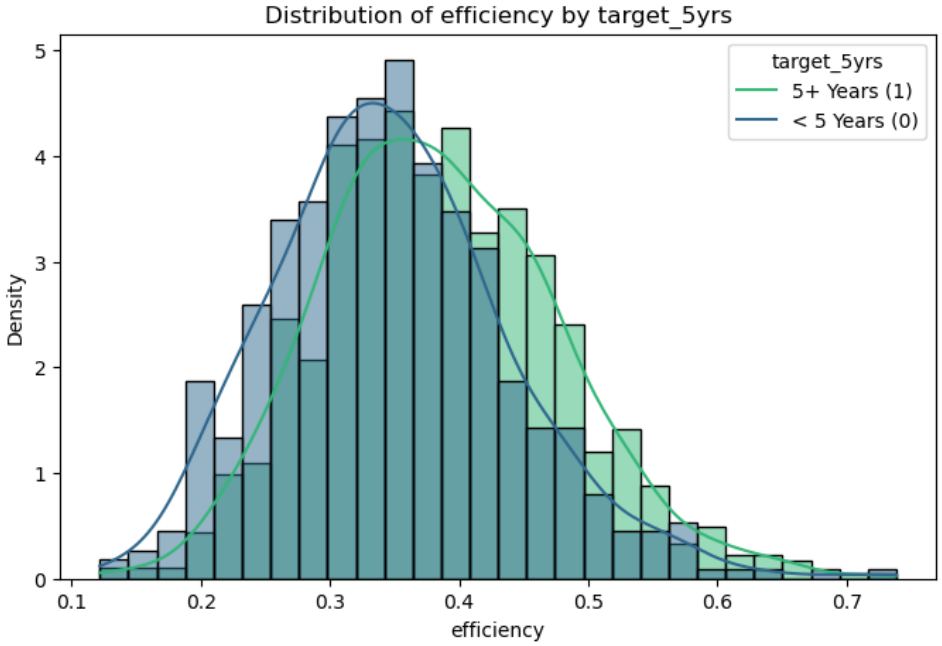

Distribution of stl by target_5yrs
target_5yrs
5+ Years (1)
< 5 Years (0)
Density
stl
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00
0.0
0.5
1.0
1.5
2.0
2.5


Distribution of blk by target_5yrs
target_5yrs
5+ Years (1)
< 5 Years (0)
Density
blk
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0


Distribution of tov by target_5yrs
target_5yrs
5+ Years (1)
< 5 Years (0)
Density
0.2
0.4
0.6
0.8
1.0
1.2
1.4


0.0
0
1
2
3
4
tov


Distribution of total_points by target_5yrs
target_5yrs
5+ Years (1)
< 5 Years (0)
0.0035
0.0030
0.0025
0.0020
0.0015
0.0010
0.0005
0.0000
Density
0
500
1000
1500
2000
total_points


Distribution of efficiency by target_5yrs
target_5yrs
5+ Years (1)
< 5 Years (0)
5
4
3
2
1
0
Density
0.1
0.2
0.3
0.4
0.5
0.6
0.7
efficiency

Most Influential Features

Based on the differences in mean values between players with short vs. long careers, the most influential features driving these longevity predictions are:

1. `total_points` **(Absolute Mean Difference: 270.62):** This is by far the strongest predictor. Players who play 5+ years accumulate significantly more total points in their initial seasons.
2. `reb` **(Rebounds, Absolute Mean Difference: 1.27):** Higher rebounding numbers are strongly associated with longer careers.
3. `ast` **(Assists, Absolute Mean Difference: 0.53):** Players with higher assist totals tend to have greater longevity.
4. `tov` **(Turnovers, Absolute Mean Difference: 0.41):** Interestingly, players with longer careers tend to have a higher average number of turnovers. This might reflect greater ball-handling responsibilities and playing time.
5. `fg` **(Field Goal Percentage, Absolute Mean Difference: 2.87):** Better shooting efficiency contributes to career length.

`3p` (3-point percentage) showed almost no difference between the two groups, suggesting it's not a strong distinguishing factor for longevity in this model.

When to Trust (and Question) Model Outputs

Trust the Model When:

* **Identifying High-Confidence Prospects:** Given the high precision (80%), when the model strongly predicts a player will have a long career, there's a good chance it's correct. These are players who align well with the statistical profile of long-tenured players, particularly excelling in `total_points`, `rebounds`, and `assists`.
* **Focusing on Core Statistical Performance:** The model highlights fundamental statistical categories like scoring, rebounding, and playmaking (`total_points`, `reb`, `ast`) as key drivers of longevity. This reinforces traditional scouting wisdom.

Question the Model When:

* **Evaluating Borderline or Unique Prospects:** Due to lower recall, the model might undervalue players who don't fit the established statistical mold but still possess valuable skills (e.g., exceptional defense not fully captured by `stl` or `blk`, or highly specialized roles). These might be "missed talents" the model flags for short careers.
* **Overlooking Contextual Factors:** The model is purely statistical and does not account for qualitative factors like leadership, work ethic, coachability, injury history, or fit within a team system. These are crucial elements for actual player longevity and should be weighed by scouts.
Features with Low Influence: *Features like `3p` showed minimal influence in this model. While important in modern basketball, this model suggests it's not a primary differentiator for longevity based on means. This doesn't mean the feature is unimportant in general, but rather that its impact on this specific longevity prediction by* this model* is limited.

Actionable Recommendations for Scouting

1. **As a First-Pass Filter:** Use this model as a preliminary screening tool to quickly identify players with a high statistical probability of long careers. This can help prioritize scouting efforts.
2. **Combine with Expert Opinion:** Never replace human scouting. Use the model's predictions as one data point among many. If the model flags a player for a short career but scouts are highly enthusiastic, conduct further in-depth analysis.
3. **Investigate Discrepancies:** When the model's prediction strongly contradicts a scout's assessment, use it as an opportunity to delve deeper. Is the player an outlier? Does the model miss a unique skill? Or is the scout's judgment potentially biased?
4. **Monitor Key Metrics:** Pay close attention to `total_points`, `rebounds`, and `assists` in prospective players, as these were identified as the most powerful statistical indicators of longevity by the model.
5. **Consider Model Refinement:** For future iterations, explore more sophisticated models (e.g., Logistic Regression, Random Forests, Gradient Boosting) that do not assume feature independence and can capture more complex relationships. Additionally, incorporate more granular data if available (e.g., minutes played, advanced metrics, college stats, qualitative scouting reports) to enhance predictive power and provide a more holistic view.
